Practical 7A

Aim: Develop a PWA that works in offline mode using Service Worker.

Step 1: Create ASP.NET Core MVC Project

1. Open Visual Studio 2026.

2. Click Create a new project.

3. Search for ASP.NET Core Web App (Model-View-Controller).

4. Select the project template and click Next.

5. Enter Project Name: SWOfflinePWA.

6. Select Framework: .NET 10, then click Create.

Step 2: Create Home Page

1. Open Views → Home → Index.cshtml.

2. Replace the existing code with:

```
<div class="text-center">
    <h1>College Information Portal</h1>
    <h3>PWA Offline Demo</h3>
    <p>This application works without internet connection.</p>
</div>
```

3. Save the file.

Step 3: Create Manifest File

1. Right-click wwwroot → Add → New Item.

2. Select JSON File and name it manifest.json.

3. Enter the following code:

```
{
  "name": "Offline PWA",
  "short_name": "OfflinePWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "white",
  "theme_color": "blue",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

4. Save the file.

Step 4: Create Icons Folder

1. Right-click wwwroot → Add → New Folder.

2. Name the folder icons.

3. Add icon1.png (192 × 192 pixels) and icon2.png (512 × 512 pixels) inside it.

Step 5: Create Service Worker

1. Right-click wwwroot → Add → New Item → JavaScript File.

2. Name the file service-worker.js.

3. Enter the following code:

```
const CACHE_NAME = "pwa-cache-v1";
const urlsToCache = ["/", "/Home/Index"];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => cache.addAll(urlsToCache))
    );
});

self.addEventListener("fetch", event => {
    event.respondWith(
        caches.match(event.request)
            .then(response => response || fetch(event.request))
    );
});
```

4. Save the file.

Step 6: Register Manifest

1. Open Views → Shared → _Layout.cshtml.

2. Add this line inside the <head> section:

```
<link rel="manifest" href="/manifest.json">
```

3. Save the file.

Step 7: Register Service Worker

1. In the same _Layout.cshtml file, add the following script before </body>:

```
<script>
    if ('serviceWorker' in navigator) {
        navigator.serviceWorker.register('/service-worker.js')
            .then(() => console.log('Service Worker Registered'));
    }
</script>
```

2. Save the file.

3. Open Program.cs and ensure app.UseStaticFiles(); exists before app.MapControllerRoute(...). Add it if missing.

Step 8: Run Application

1. Run the project in Visual Studio.

2. Open the application in Google Chrome.

3. Check whether the PWA installation icon appears in the browser.

4. Press F12 to open Developer Tools.

Step 9: Verify Service Worker

1. Open the Application tab in Developer Tools.

2. Select Service Workers from the left panel.

3. Check that service-worker.js is registered and its status is activated and running.

Step 10: Test Offline Mode

1. Keep the application open in Chrome.

2. Press F12 and select the Network tab.

3. Enable the Offline option.

4. Refresh the webpage.

5. Verify that the cached webpage opens without an internet connection.

Result

A Progressive Web Application (PWA) was developed using ASP.NET Core MVC and a Service Worker. The application was tested in offline mode.

Note: Serve the application over HTTPS or localhost. The pages must be successfully cached before going offline. If caching fails, check that the listed routes return successful responses.